

Filas de Prioridades

Estruturas de Dados

Prof. Dr. Anderson Bessa da Costa

Universidade Federal de Mato Grosso do Sul

Introdução

- Em muitas aplicações, existe uma **prioridade** associada a cada dado
 - Determinar repetidas vezes o dado de maior **prioridade** pode ser uma operação muito importante

Exemplo: Lista de Tarefas

 Inbox 5

 Today 13

 Upcoming

▼

Favorites

 Today's Objectives 2

 Today's Focus 7

 Today's Communications 4

 Tomorrow's Work 12

>

Projects

>

Labels

>

Filters

Tomorrow's Work

↓↑ Sort

Today · Wed May 19

Exercise

↺↻

Home / Daily ●

Send quote to Phil

THIS WEEK ●

Review and update client list

ROUTINES ●

Record podcast

THIS WEEK ●

Business development

THIS WEEK ●

Call Sarah about virtual conference

THIS WEEK ●

Follow up with Fred about launch

THIS WEEK ●

Tomorrow · Thu May 20

Finish editing Q2 report for Friday's all-hands

deadline

Work 👤

Create presentation for launch

🔖 0/5 💬 1

Work 👤

Exemplo: Lista de Tarefas (Cont.)

- Deseja-se que as tarefas sejam realizadas em ordem decrescente de prioridades
- Prioridades das tarefas podem variar ao longo do tempo
- Novas tarefas podem ingressar na tabela

Filas de Prioridades

- **Fila de prioridades** é uma tabela na qual a cada um de seus dados está associada uma **prioridade**
- Essa **prioridade** é, em geral, definida através de um valor numérico e armazenada em algum de seus campos

Filas de Prioridades: Operações

- As operações básicas são:
 - seleção do elemento de maior prioridade;
 - inserção de um novo elemento;
 - remoção do elemento de maior prioridade.
- Frequentemente, deseja-se também efetuar a alteração da prioridade

Implementação de Filas de Prioridades

- Serão abordadas três implementações distintas:
 - Por lista não ordenada
 - Por lista ordenada
 - Por heap
- As referências são as operações básicas de *seleção, inserção, remoção, alteração e construção da tabela*

Implementação por Lista Não Ordenada

- **inserção** e construção triviais
 - O novo nó da tabela pode ser colocado em qualquer posição conveniente, dependendo do tipo de alocação utilizada, sequencial ou encadeada
- **remoção** implica percorrer a tabela em busca do elemento de maior prioridade
- **alteração de prioridade** não afeta a organização da tabela, mas pressupõe a busca do elemento
- O mesmo acontece com a operação de **seleção** do elemento de maior prioridade

Implementação por Lista Não Ordenada (Cont.)

- Para uma tabela com n elementos, cada uma dessas operações requer o seguinte número de passos:
 - seleção $\Rightarrow O(n)$
 - inserção
 - $O(1)$
 - remoção
 - $O(n)$
 - alteração
 - $O(n)$
 - construção
 - $O(n)$

Implementação por Lista Ordenada

- **remoção** é imediata porque, estando as prioridades já ordenadas, o primeiro elemento é o que interessa
- **inserção** obriga a um percurso pela lista para procurar sua posição correta
- **alteração de prioridade** é semelhante a uma nova inserção
- **seleção** também é imediata
- **construção** exige uma ordenação prévia da tabela

Implementação por Lista Ordenada (Cont.)

- Então, cada uma das operações requer o seguinte número de passos:
 - seleção $\Rightarrow O(1)$
 - inserção
 - $O(n)$
 - remoção
 - $O(1)$
 - alteração
 - $O(n)$
 - construção
 - $O(n \log n)$

Heap

- As duas implementações anteriores apresentam a inconveniência de possuírem operações de atualização de complexidade $\Omega(n)$
- **Heap** é uma implementação mais eficiente
- **Heap** é uma lista linear composta de elementos com chave $s_0, \dots, s_{n-1}$ satisfazendo a seguinte propriedade:

$$s_i \leq s_{\lfloor (i-1)/2 \rfloor}, \text{ para } 0 < i < n \quad (i)$$

Heap: Propriedade

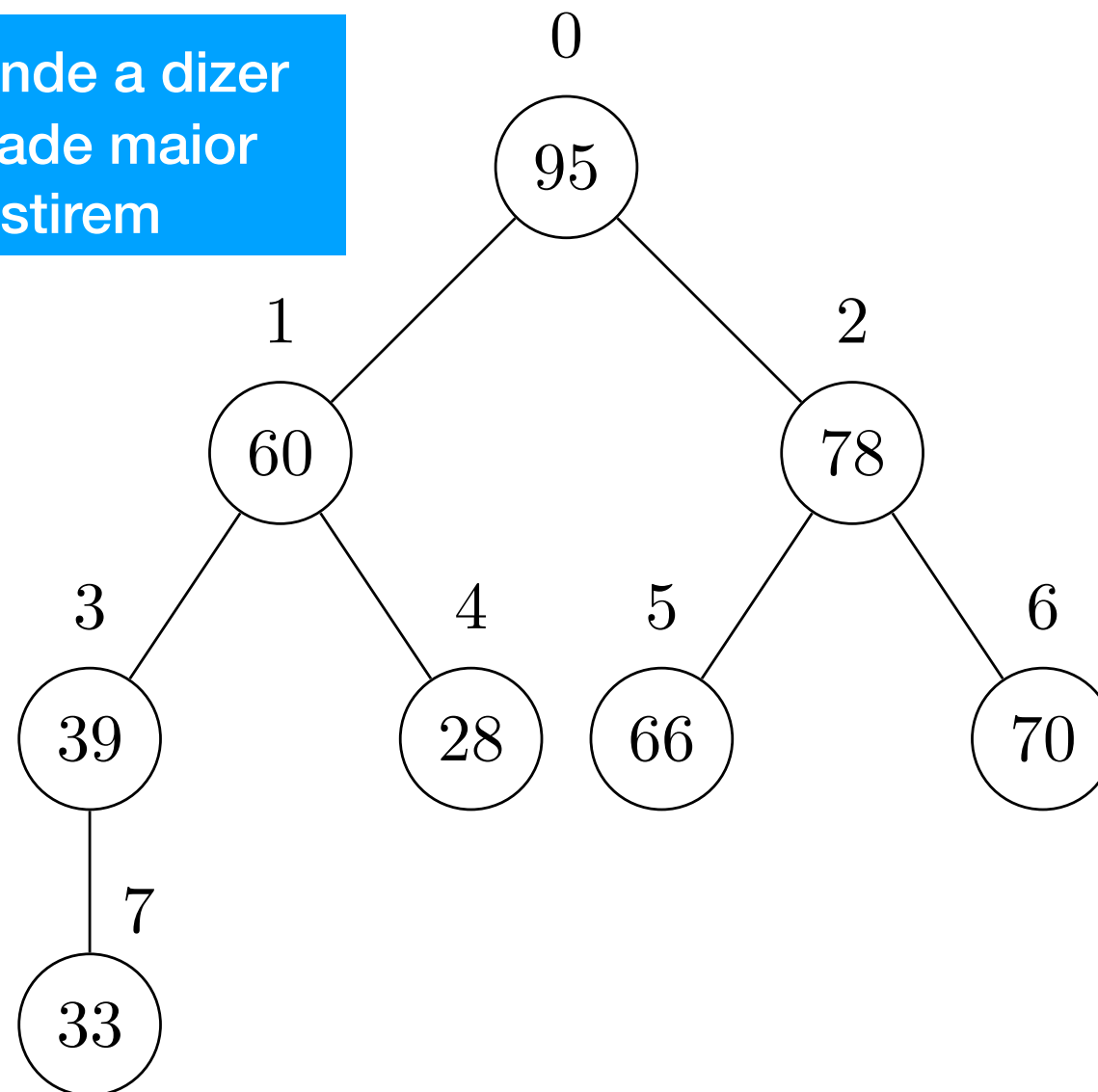
- Por exemplo, a lista apresentada abaixo obedece à propriedade (*i*)

0	1	2	3	4	5	6	7	8	9
95	60	78	39	28	66	70	33		

Heap: Representação Gráfica

A visualização desta propriedade é facilitada quando se imagina a tabela disposta numa árvore binária.

A propriedade (i) corresponde a dizer que cada nó possui prioridade maior que seus dois filhos, se existirem



Implementação por Heap

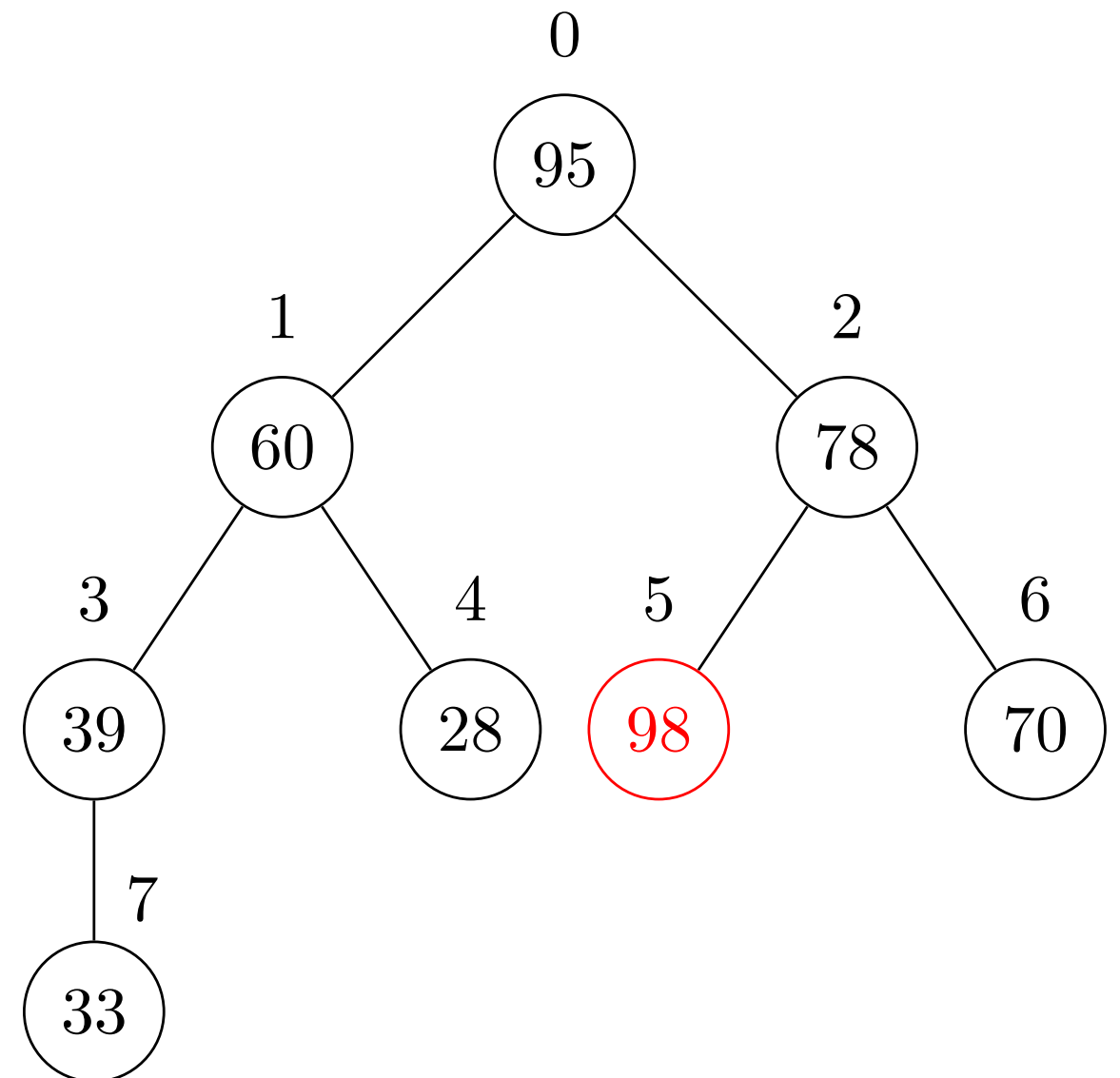
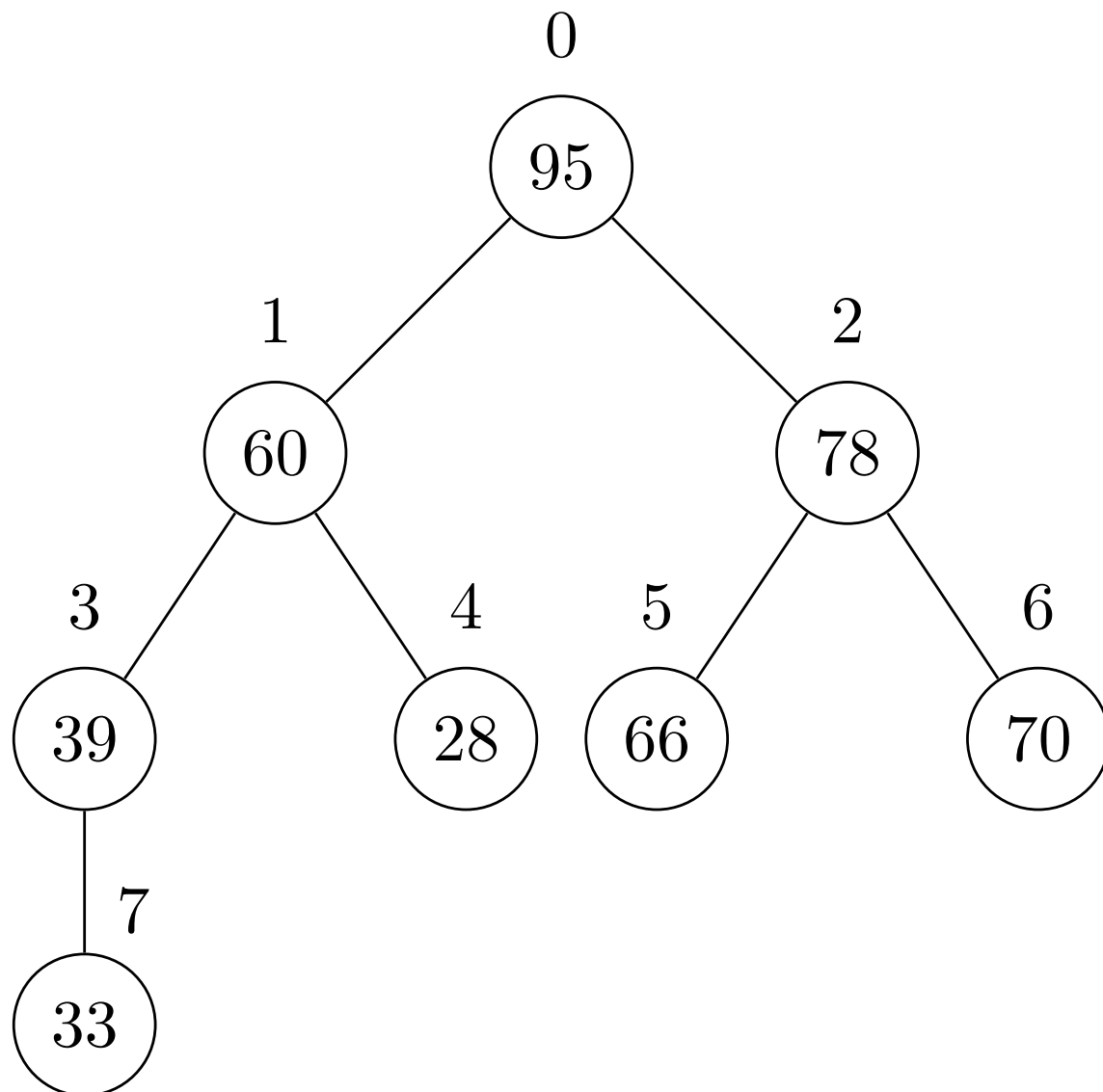
- A condição (i) implica que o elemento de maior prioridade seja sempre o primeiro da ordenação, isto é, a raiz da árvore
 - Então, a operação de **seleção** é trivial
- As operações de **inserção**, **remoção** e **alteração** podem ser realizadas em tempo $(\log n)$
- A **construção** de um *heap* requer não mais do que tempo $O(n)$

Implementação por Heap (Cont.)

- Então, são os seguintes os parâmetros indicadores de eficiência correspondentes a um **heap**:
 - seleção $\Rightarrow O(1)$
 - inserção
 - $O(\log n)$
 - remoção
 - $O(\log n)$
 - alteração
 - $O(\log n)$
 - construção
 - $O(n)$

Alteração Prioridade: Aumentar

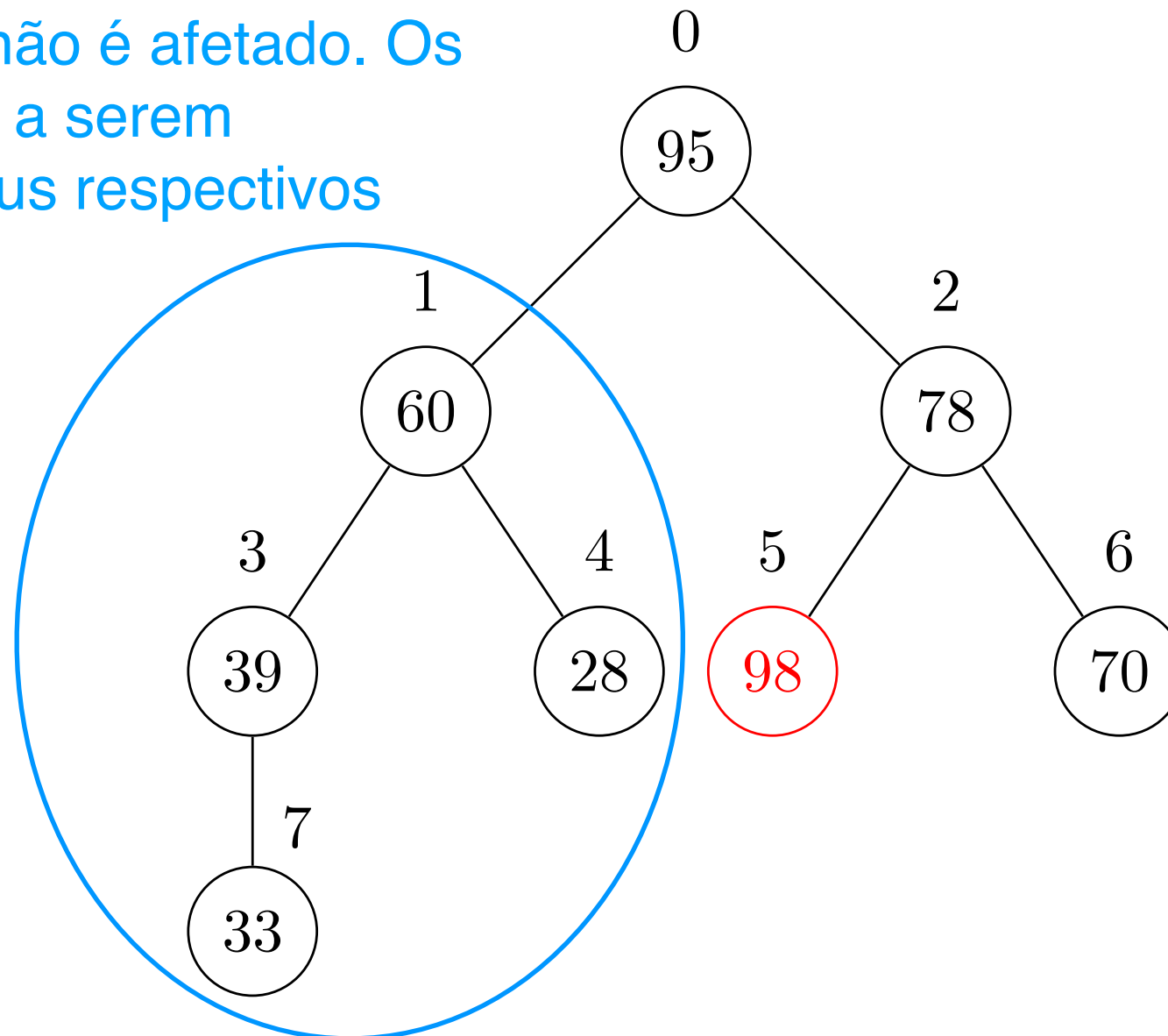
Considere o **heap** representada abaixo. Desejamos alterar a prioridade do nó 5 de 66 para 98:



Alteração Prioridade: Aumentar (2)

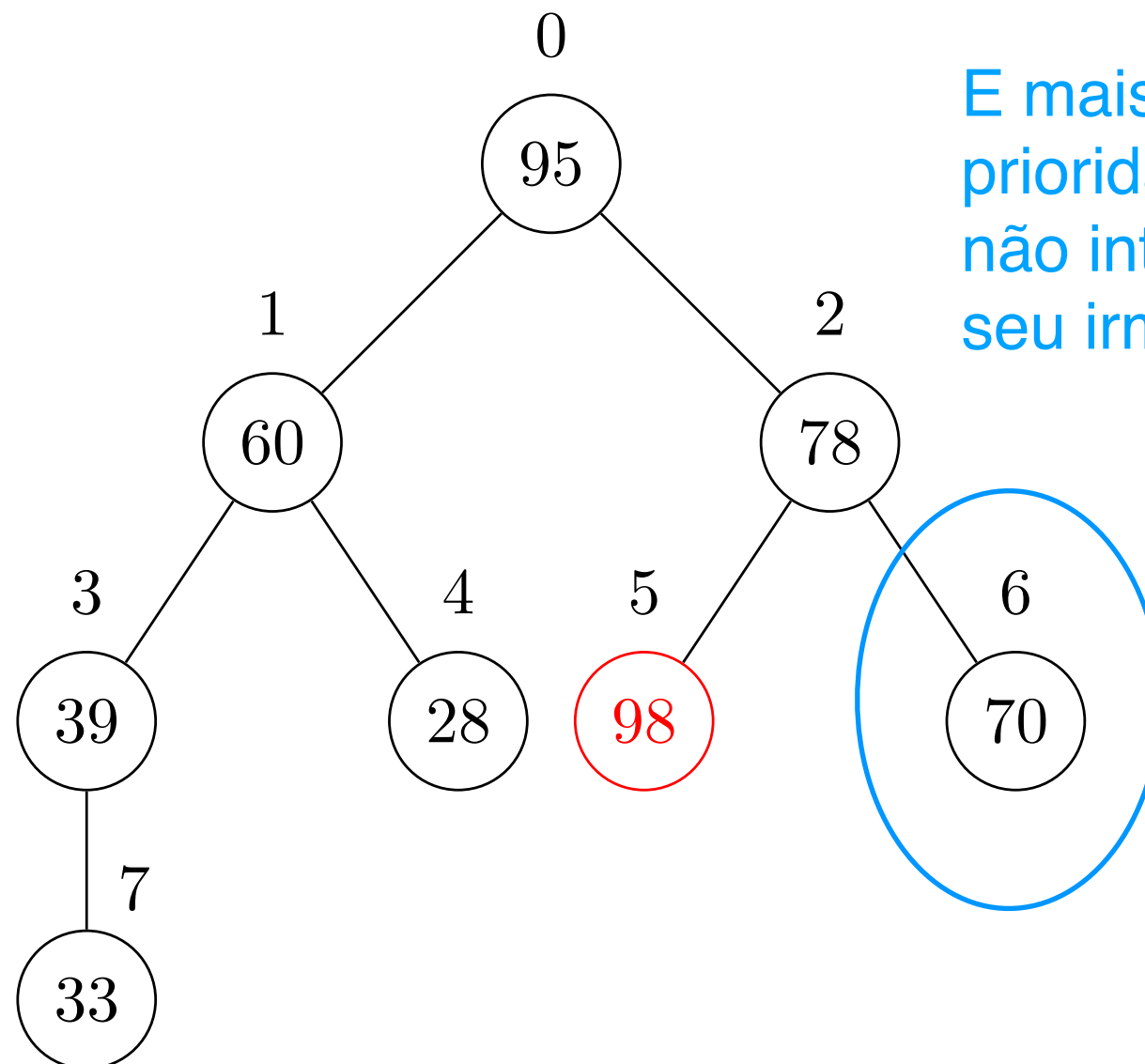
Considere o **heap** representada abaixo. Desejamos alterar a prioridade do nó 5 de 66 para 98:

Lado esquerdo não é afetado. Os filhos continuam a serem menores que seus respectivos pais.



Alteração Prioridade: Aumentar (3)

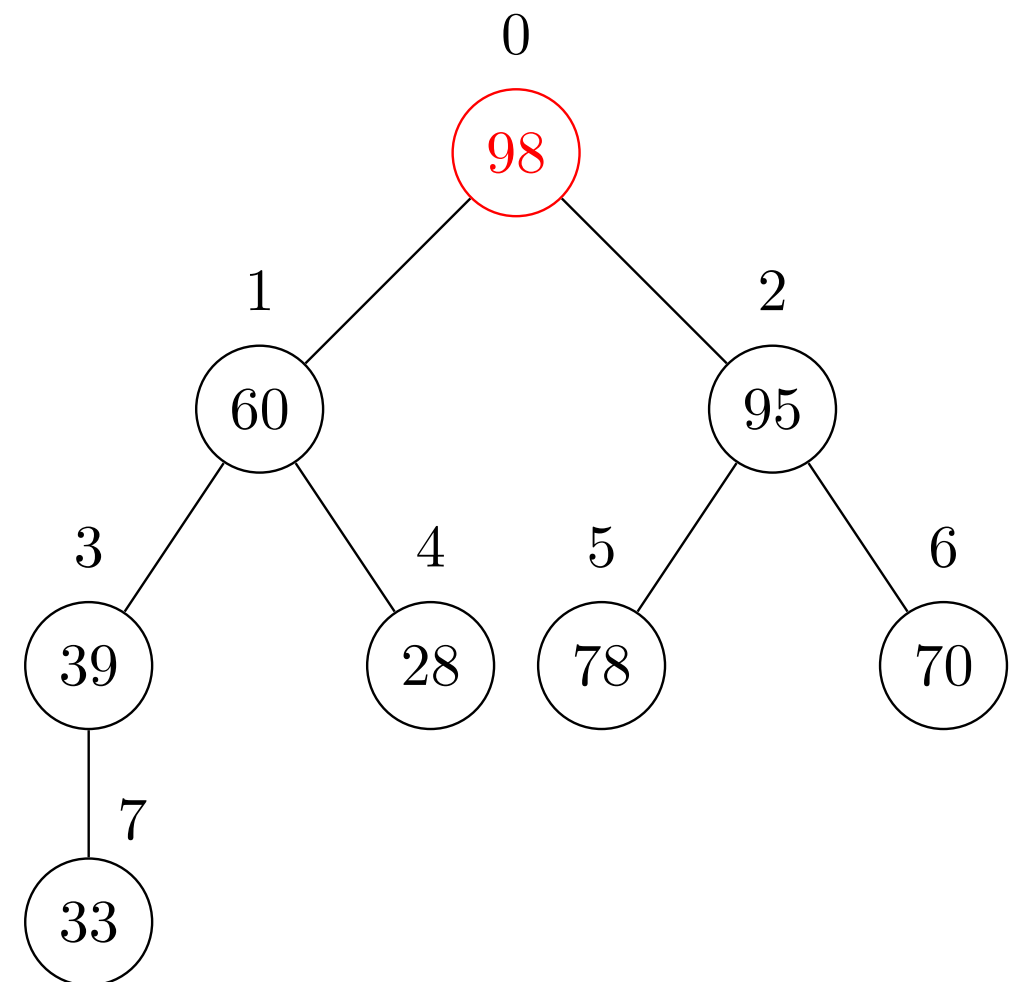
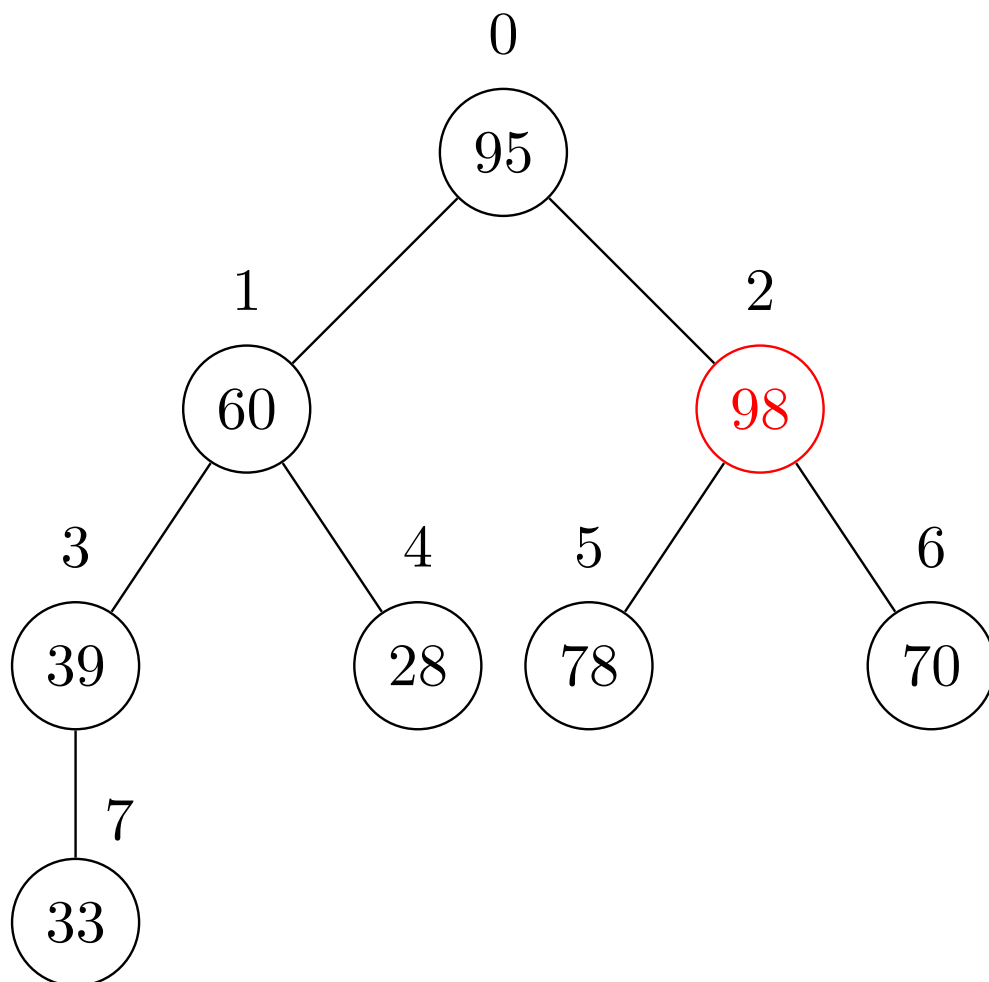
Considere o **heap** representada abaixo. Desejamos alterar a prioridade do nó 5 de 66 para 98:



E mais: o aumento de prioridade do nó 5 também não interfere na posição de seu irmão na árvore, o nó 6.

Alteração Prioridade: Aumentar (4)

O elemento alterado será então alocado na posição correta simplesmente **subindo** pelo caminho que leva à raiz da árvore através de trocas, até que a propriedade seja satisfeita

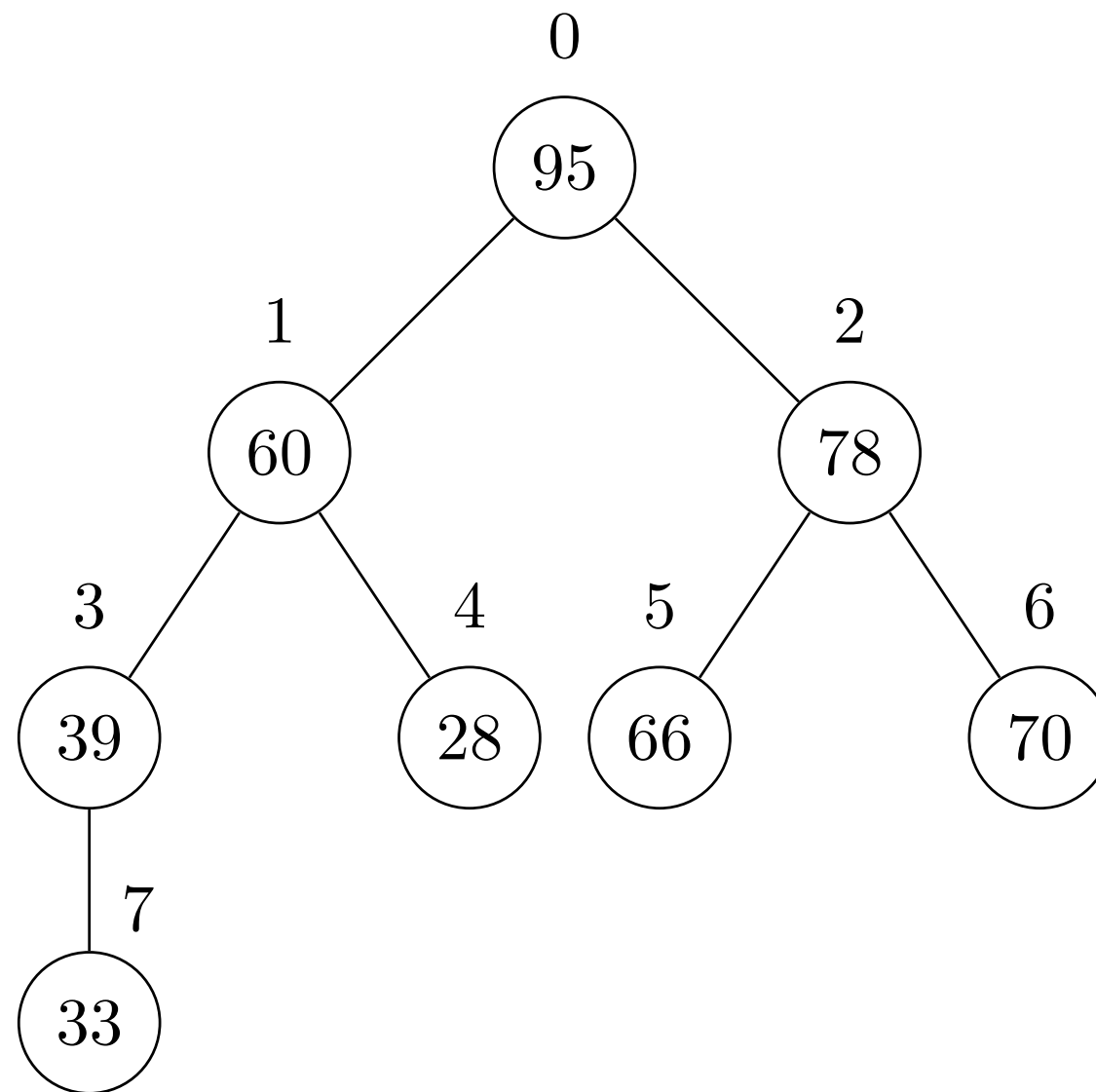


Algoritmo Subir

```
1. // Subir por um caminho da arvore
2. void suba(int h[], int i) {
3.     int j;
4.
5.     // se i tem pai
6.     if(i > 0) {
7.         // j recebe o indice do pai
8.         j = pai(i);
9.
10.        // se prioridade de i for maior que prioridade de pai de i (j)
11.        if(h[i] > h[j]) {
12.            // troque i com o pai de i (j)
13.            troque(&h[i], &h[j]);
14.            // chame recursivamente subir passando o indice do pai de i (j)
15.            suba(h, j);
16.        }
17.    }
18.}
```

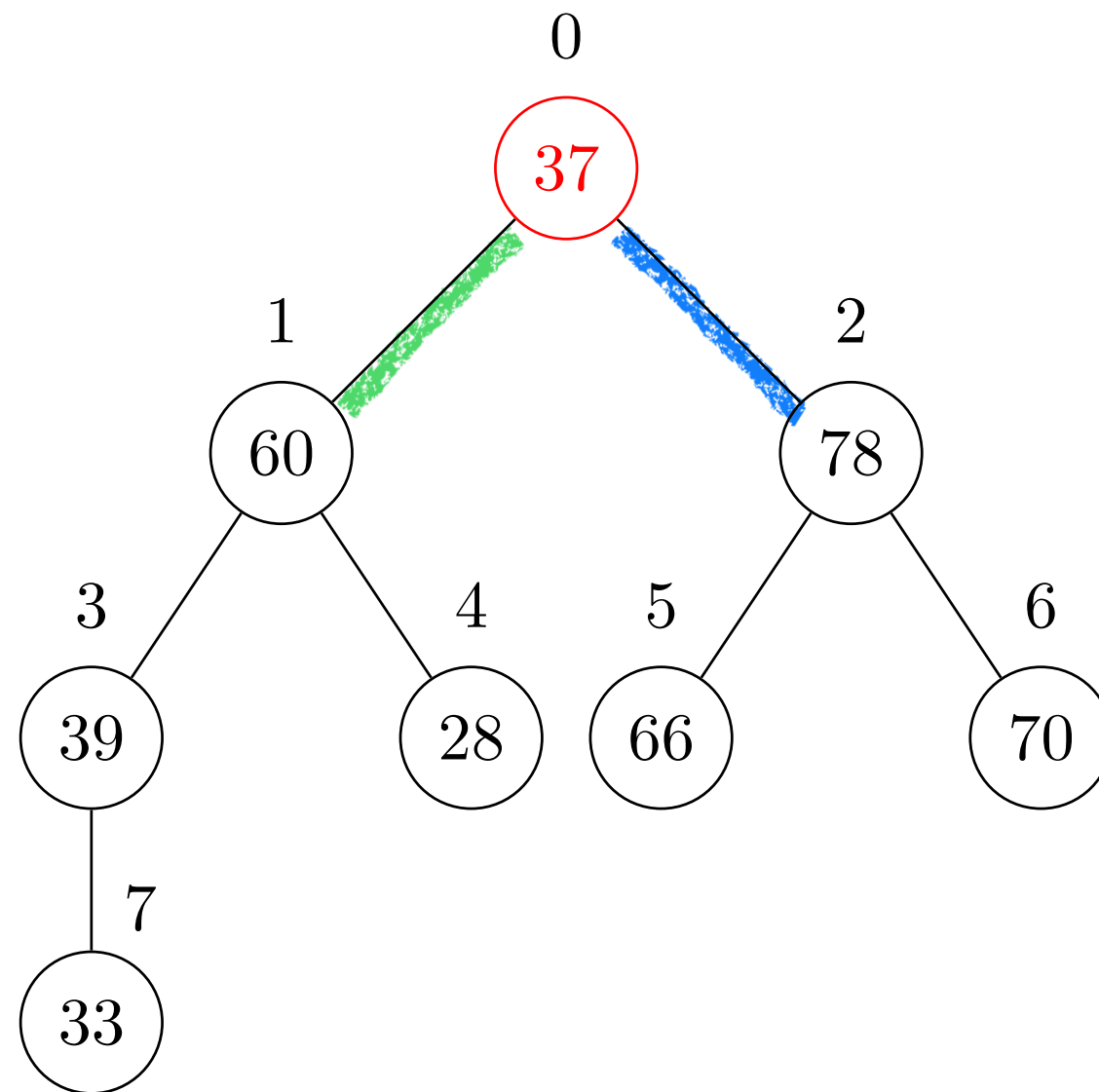
Alteração Prioridade: Diminuir

Desejamos alterar a prioridade do nó 0 de 95 para 37.



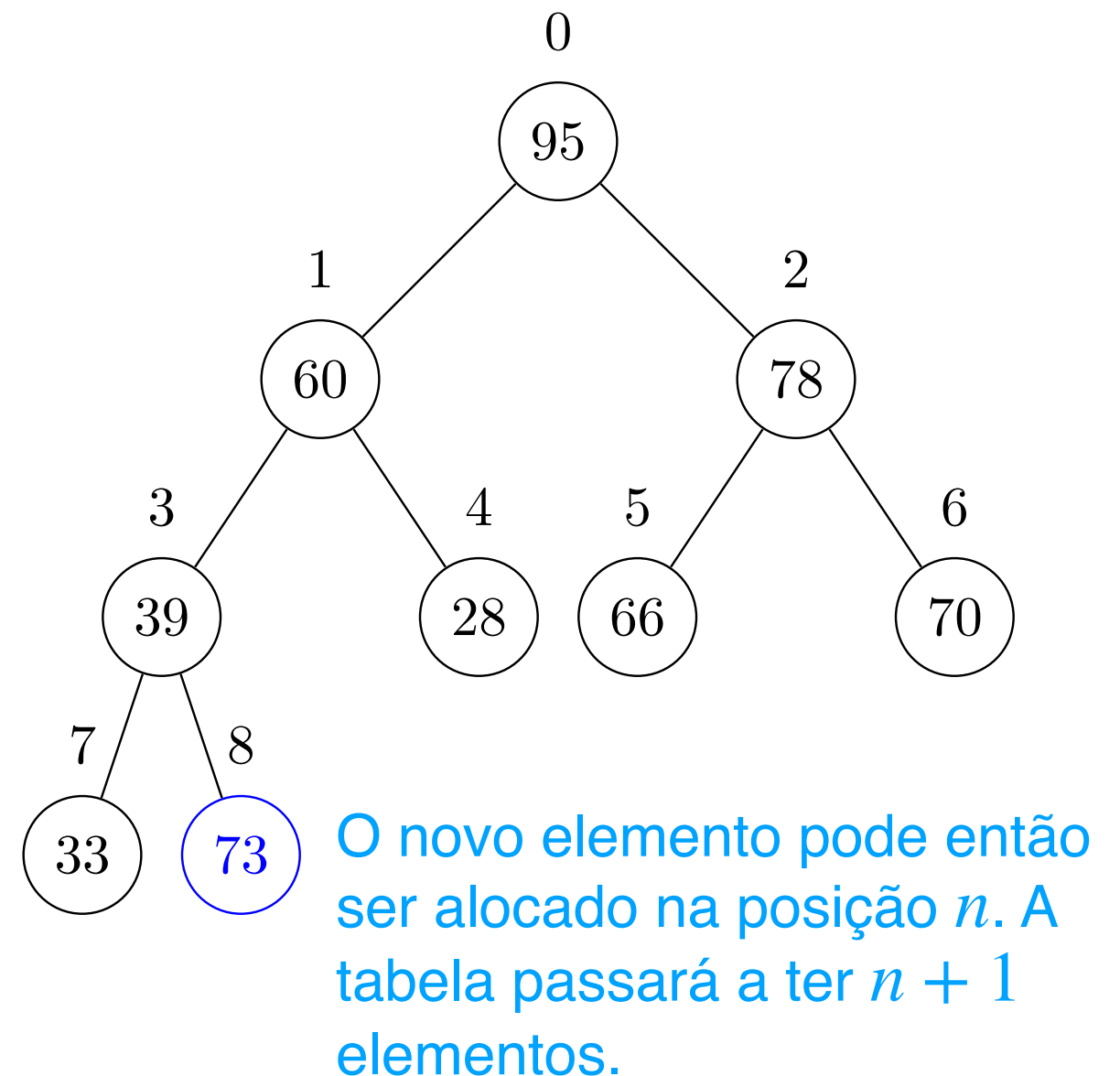
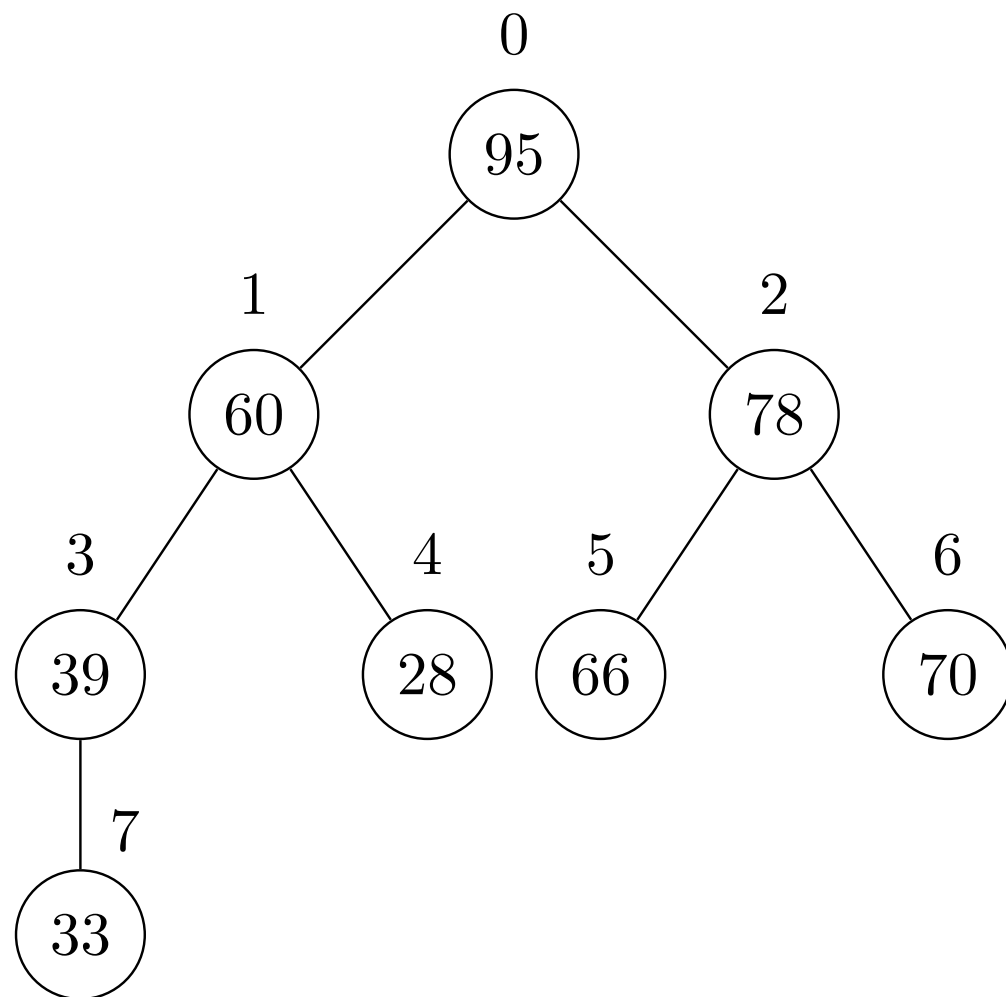
Alteração Prioridade: Diminuir (2)

Descer pela esquerda ou direita?



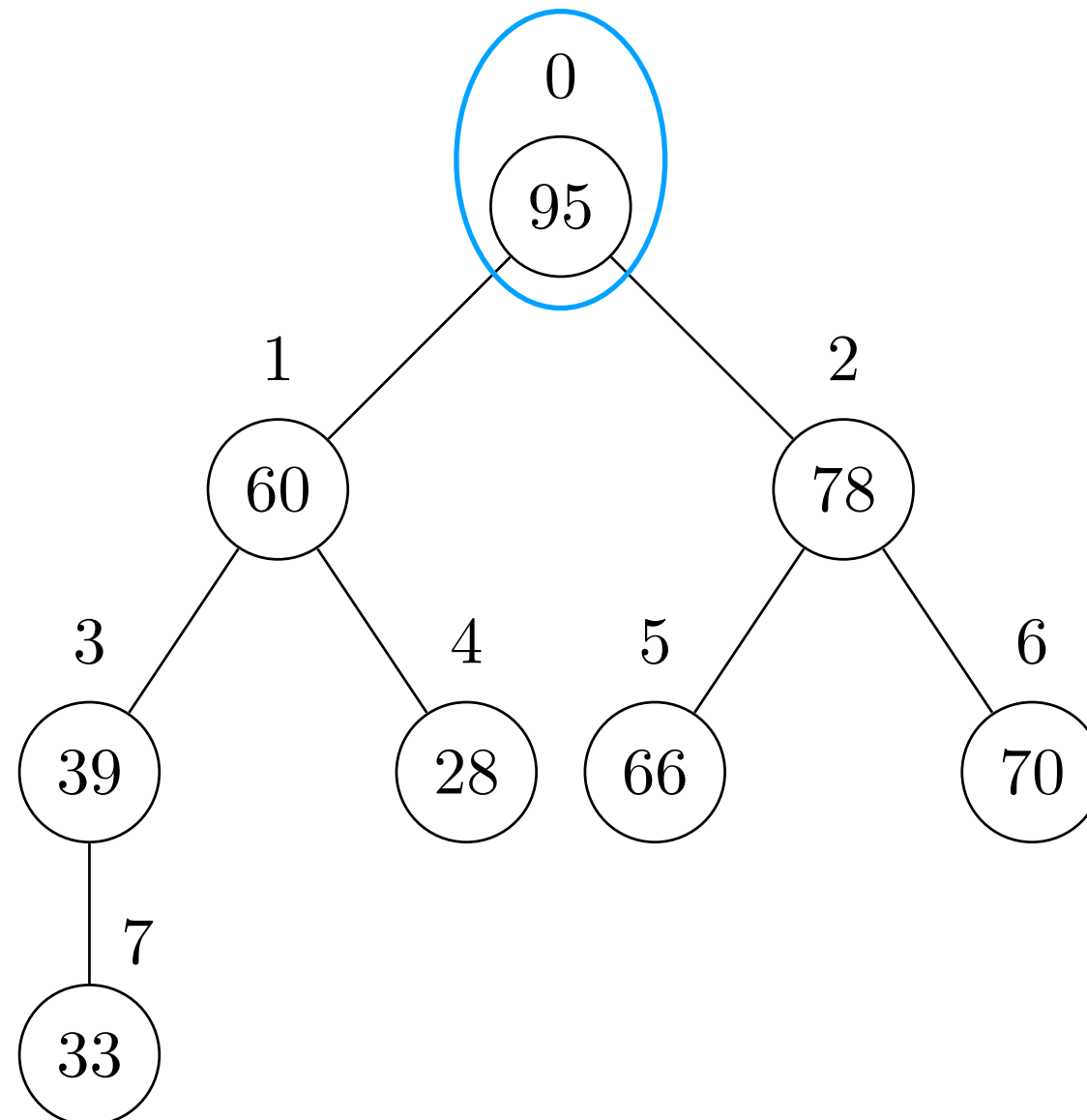
Inserção

Suponha a **inserção** de um novo elemento de prioridade 73.



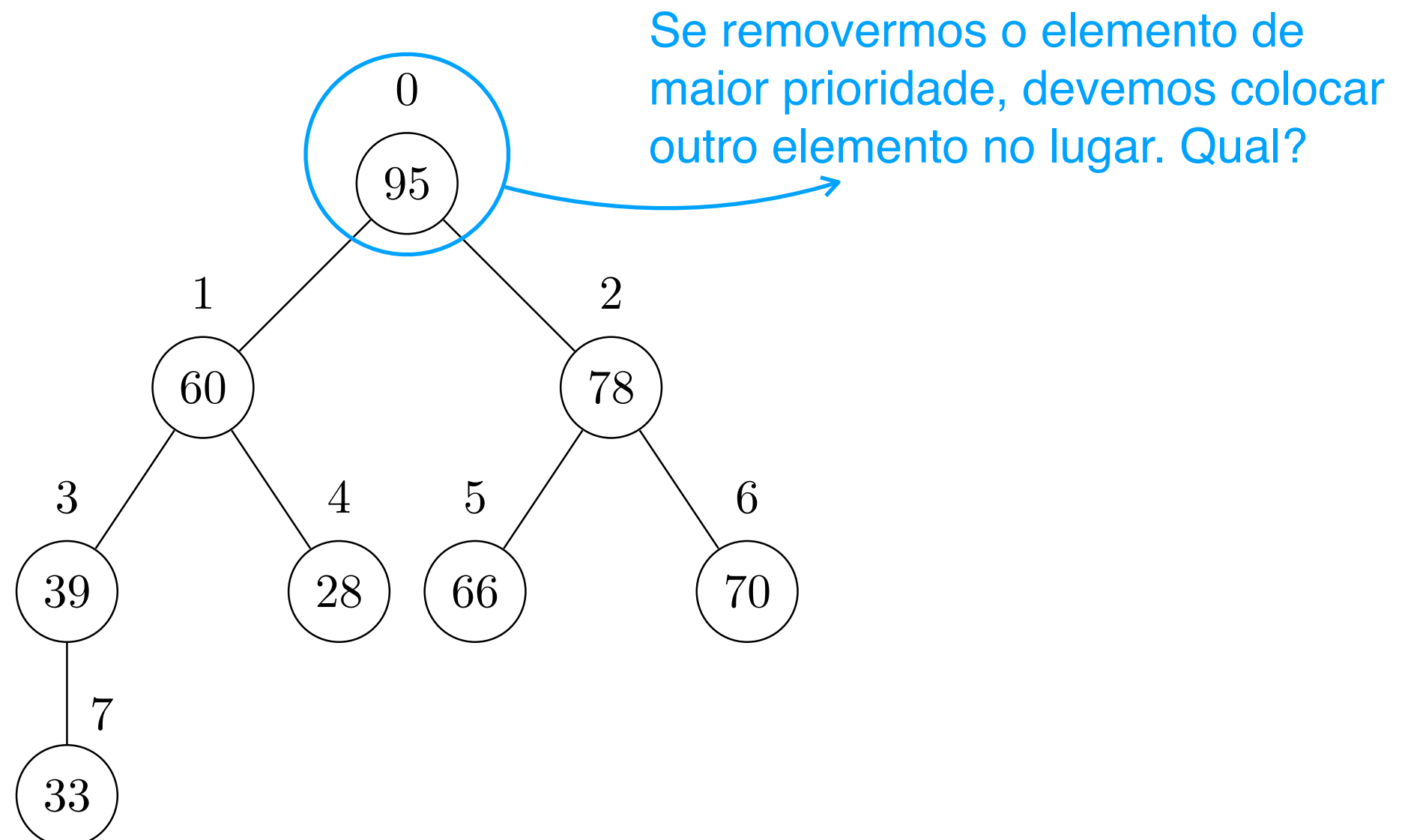
Seleção

O elemento de maior prioridade sempre se encontra na primeira posição (índice **0**).



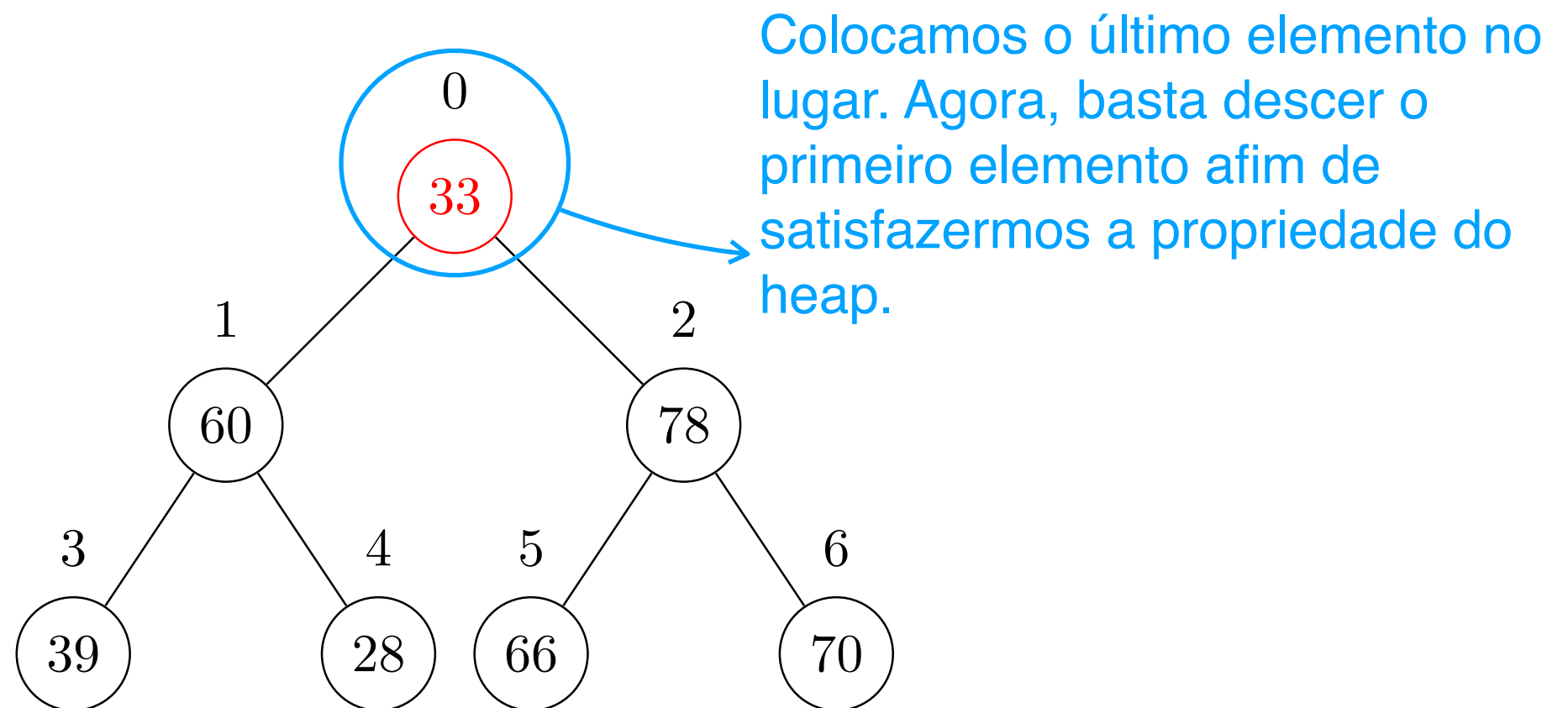
Remoção

- A **remoção** do elemento de maior prioridade, o primeiro na representação utilizada, deixa sua posição vazia, que deve então ser preenchida.



Remoção (Cont.)

- A **remoção** do elemento de maior prioridade, o primeiro na representação utilizada, deixa sua posição vazia, que deve então ser preenchida.



Construção

- Uma primeira solução provém da observação de que toda lista ordenada corresponde a um **heap**
- Assim, um **heap** pode ser construído simplesmente através da ordenação de uma lista
- Complexidade $O(n \log n)$

Construção

- Observe que a propriedade é sempre satisfeita quando se trata de um nó sem filhos, isto é, nós alocados a partir da posição $\text{piso}(n/2) + 1$
 - Complexidade $O(n)$

```
1. void construa (int t[], int n) {  
2.     for (i = n/2-1; i >= 0; i--) {  
3.         desca(t, i, n);  
4.     }  
5. }
```

Referências

- CORMEN, T. H..[et al]. Algoritmos: teoria e prática. 3^a ed. Rio de Janeiro: Elsevier, 2012.
- SZWARCFITER, Jayme Luiz; MARKENZON, Lilian. Estruturas de Dados e seus Algoritmos. Edição: 3a. Editora: LTC. 2010